

AI ENGINEERING — WEEK 3

Lambda, NumPy & Pandas: Data Foundations for AI Engineering

Lambda functions, vectorized NumPy, Pandas data cleaning, vectors & similarity, and train/test splitting — the data-handling foundation behind everything from cleaning a fine-tuning dataset to powering semantic search and RAG retrieval.

Builds directly on Week 1 (Python Foundations) and Week 2 (OOP, Functions, Errors & Git). Lab exercises are not included here — see the separate lab handout.

Chapter 1 — Lambda Functions for Data Work

By the end of this chapter you should be able to

- Write a lambda function and explain when it's appropriate to use one
- Use lambda with `sorted()`, `map()`, and `filter()`
- Recognize where lambda fits into the Pandas and NumPy work later in this guide

Connecting back: Week 2, Chapter 4 established that functions are first-class citizens in Python — they can be passed around like any other value — and introduced higher-order functions (functions that take other functions as arguments), like `sorted(..., key=...)`. Lambda is simply the syntax for writing a small, throwaway function *inline*, right at the point you need to pass one to a higher-order function, without a separate `def` block.

1.1 Lambda Syntax

```
lambda arguments: expression
```

A lambda is an anonymous function limited to a single expression — no statements, no multiple lines, and the result of that expression is automatically returned.

```
square = lambda x: x ** 2
print(square(5))    # 25

add = lambda a, b: a + b
print(add(3, 4))    # 7
```

`square = lambda x: x ** 2` is functionally identical to:

```
def square(x):
    return x ** 2
```

Lambda doesn't do anything a regular function can't — it's purely a shorthand for cases where defining a whole named function would be overkill.

1.2 Lambda with `sorted()`

The `key` parameter of `sorted()` takes a function that tells Python what value to sort by. Lambda is the natural fit here since the sort key is usually a small, one-off expression.

```
students = [
    {"name": "Amara", "grade": 92},
    {"name": "Leo", "grade": 85},
    {"name": "Priya", "grade": 78},
]

by_grade = sorted(students, key=lambda s: s["grade"], reverse=True)
print([s["name"] for s in by_grade])    # ['Amara', 'Leo', 'Priya']
```

1.3 Lambda with `map()` and `filter()`

```
nums = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x ** 2, nums))
print(squared)    # [1, 4, 9, 16, 25]

evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)    # [2, 4]
```

A list comprehension is usually considered more Pythonic than `map()` / `filter()` with lambda for simple cases: `[x**2 for x in nums]` reads at least as clearly as the `map()` version above. Reach for `map()` / `filter()` / lambda mainly when you're passing the function into something else that expects one — like `sorted()`, or, as you'll see shortly, Pandas' `.apply()`.

1.4 A Preview: Lambda Inside Pandas

The single most common place you'll use lambda in this course is inside a Pandas `.apply()` call, transforming every value in a column with a small custom rule:

```
# a quick preview – the full Pandas treatment is in Chapter 3
import pandas as pd
df = pd.DataFrame({"score": [55, 82, 91]})
df["passed"] = df["score"].apply(lambda x: "Yes" if x >= 60 else "No")
print(df)
#    score passed
# 0     55     No
# 1     82     Yes
# 2     91     Yes
```

1.5 When NOT to Use Lambda

Readability limit: if the logic needs more than one expression, a conditional with multiple branches, or a name that documents what it does, write a proper `def` function instead and pass that in by name.

```
# hard to read – resist this
result = sorted(records, key=lambda r: (r["dept"], -r["salary"], r["name"].lower()))

# clearer – give it a name
def sort_key(r):
    return (r["dept"], -r["salary"], r["name"].lower())

result = sorted(records, key=sort_key)
```

Where You'll Actually Use This

Cleaning a fine-tuning dataset — filtering out examples that are too short, too long, or otherwise unusable before they ever reach a training run:

```
examples = [
    {"text": "Hi"},
    {"text": "Explain how photosynthesis works in plants, step by step."},
    {"text": "x" * 5000}, # way too long – probably a scraping error
]

clean_examples = list(filter(lambda ex: 20 < len(ex["text"]) < 2000, examples))
print(len(clean_examples)) # 1 – only the middle example survives
```

Ranking retrieved documents — sorting RAG or search results by relevance score before showing the top matches:

```
results = [
    {"chunk": "Refunds take 3-5 business days.", "score": 0.71},
    {"chunk": "Our return policy allows 30 days.", "score": 0.92},
    {"chunk": "Contact support for order issues.", "score": 0.58},
]

top_results = sorted(results, key=lambda r: r["score"], reverse=True)
print(top_results[0]["chunk"]) # 'Our return policy allows 30 days.' – highest score first
```

Quick inline transforms — reformatting a column of raw API responses without breaking your flow to define a one-off named function:

```
raw_responses = [{"user_id": 1, "msg": "hello"}, {"user_id": 2, "msg": "hi there"}]
formatted = list(map(lambda r: f"User {r['user_id']}: {r['msg']}", raw_responses))
print(formatted) # ['User 1: hello', 'User 2: hi there']
```

Chapter 2 — NumPy Arrays & Vectorization

By the end of this chapter you should be able to

- Create, index, slice, and reshape NumPy arrays
- Explain broadcasting and predict when two shapes are compatible
- Write vectorized code instead of a Python loop, and explain why it's faster
- Recognize where vectorization does and doesn't apply

2.1 Why NumPy

A Python list can hold anything, which means each element carries overhead and elements aren't guaranteed to sit next to each other in memory. A NumPy array holds one fixed data type in one contiguous memory block, which lets NumPy hand off bulk operations to fast, compiled C code instead of looping in Python one element at a time. This is the entire reason NumPy exists: the same math, done far faster on large data.

2.2 Creating Arrays

```
import numpy as np

a = np.array([1, 2, 3, 4])
zeros = np.zeros(5)           # array([0., 0., 0., 0., 0.])
ones = np.ones((2, 3))        # 2x3 array of ones
rng = np.arange(0, 10, 2)      # array([0, 2, 4, 6, 8]) – like range(), but an array
lin = np.linspace(0, 1, 5)     # array([0., 0.25, 0.5, 0.75, 1.]) – 5 evenly spaced points

print(a.shape, a.dtype, a.ndim) # (4,) int64 1
```

2.3 Indexing, Slicing & Boolean Masking

```
grid = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(grid[1])                # [4 5 6] – second row
print(grid[1, 2])              # 6 – row 1, column 2
print(grid[:, 0])              # [1 4 7] – first column, all rows

nums = np.array([10, 15, 20, 25, 30])
mask = nums > 18
print(mask)                    # [False False True True True]
print(nums[mask])              # [20 25 30] – boolean masking: keep only where the mask is True
```

2.4 Reshaping & Combining Arrays

```
flat = np.arange(6)
grid = flat.reshape(2, 3)
print(grid)
# [[0 1 2]
#   [3 4 5]]

a = np.array([1, 2])
b = np.array([3, 4])
print(np.concatenate([a, b])) # [1 2 3 4]
print(np.stack([a, b]))       # [[1 2] [3 4]] – stacks as new rows
```

2.5 Broadcasting

Broadcasting lets NumPy apply an operation between arrays of different (but compatible) shapes, without you writing an explicit loop.

```
prices = np.array([10, 20, 30])
print(prices * 1.1)    # [11. 22. 33.] – a scalar "broadcasts" across every element

matrix = np.array([[1, 2, 3], [4, 5, 6]])
row = np.array([10, 20, 30])
print(matrix + row)
# [[11 22 33]
#  [14 25 36]] – row broadcasts across every row of the matrix
```

The broadcasting rule: comparing shapes from right to left, each dimension must either match exactly or be 1. A (2, 3) matrix and a (3,) row are compatible because the trailing dimension matches (3 = 3). A (2, 3) matrix and a (2,) array are **not** compatible, and NumPy raises a `ValueError`.

2.6 Vectorization in Practice

```
import numpy as np, time

n = 2_000_000
arr = np.arange(n)

start = time.time()
result = [x * 2 for x in arr]    # pure Python loop (via comprehension)
loop_time = time.time() - start

start = time.time()
result = arr * 2                # vectorized NumPy
vector_time = time.time() - start

print(f"Loop: {loop_time:.4f}s, Vectorized: {vector_time:.4f}s")
# Loop: ~0.3s, Vectorized: ~0.005s – vectorized is dramatically faster
```

2.7 Useful Vectorized Operations

```
data = np.array([[1, 2, 3], [4, 5, 6]])

print(data.sum())              # 21 – sum of everything
print(data.sum(axis=0))        # [5 7 9] – sum DOWN each column
print(data.sum(axis=1))        # [6 15] – sum ACROSS each row
print(data.mean())             # 3.5
print(np.where(data > 3, "high", "low"))
# [['low' 'low' 'low']
#  ['high' 'high' 'high']]
```

Remembering axis : `axis=0` collapses rows (moves *down* the columns), `axis=1` collapses columns (moves *across* each row). If the result's shape looks wrong, you likely have the axis backward.

2.8 Where Lambda and NumPy Meet

Applying a lambda to a NumPy array element-by-element (via a loop or plain `map()`) gives up the entire performance benefit of vectorization — you're back to one-at-a-time Python calls. Lambda's real place alongside NumPy is as a small piece of logic *inside* a vectorized call, like the condition/value branches of `np.where()` above, or as a sort key when combined with `sorted()` on structured data. "Vectorized" means letting NumPy's compiled operations act on the whole array at once — lambda is a convenience for expressing small logic, not a substitute for that.

2.9 The Honest Limits of Vectorization

Not every computation can be vectorized. If each step genuinely depends on the previous step's result (a running simulation, for example), a loop is often unavoidable — and that's fine. The skill is recognizing when vectorization applies, not forcing it everywhere.

2.10 Common Pitfalls

- **Shape mismatches:** read the `ValueError` message carefully — it names the exact shapes that failed to broadcast.
- **Forgetting axis :** a sum/mean without specifying `axis` collapses the *entire* array to one number, which is rarely what you want on 2D data.
- **Looping out of habit:** if you're writing `for i in range(len(arr))` to do elementwise math, there's almost always a

vectorized replacement.

Where You'll Actually Use This

Every embedding is a NumPy array under the hood — when a model returns a vector, it lands as (or is immediately converted to) exactly this type, ready for the math in Chapter 5:

```
embedding_from_model = [0.021, -0.314, 0.882, 0.114] # e.g. a raw list from an API response
embedding = np.array(embedding_from_model)
print(type(embedding), embedding.shape) # <class 'numpy.ndarray'> (4,)
```

Batch-processing model inputs — normalizing a whole batch of feature rows in one vectorized pass instead of looping:

```
batch = np.array([[10, 200, 3], [12, 150, 5], [8, 220, 2]], dtype=float)
normalized = (batch - batch.mean(axis=0)) / batch.std(axis=0)
print(normalized.round(2))
# each column now has mean 0 — computed for the whole batch at once, no loop
```

Fast similarity search over many vectors at once — broadcasting one query vector against every stored vector simultaneously, the core operation a vector database performs at scale:

```
stored_vectors = np.array([[1, 0, 1], [0, 1, 1], [1, 1, 0]])
query = np.array([1, 0, 1])

dot_products = stored_vectors @ query # matrix-vector product — scores all 3 at once
print(dot_products) # [2 1 1] — first stored vector matches the query best
```

Chapter 3 — Pandas Fundamentals

By the end of this chapter you should be able to

- Explain the relationship between a NumPy array, a Pandas Series, and a DataFrame
- Create a DataFrame and load one from a CSV file safely
- Inspect, select, and filter data in a DataFrame

Connecting back: a Pandas `DataFrame` is a class, and every `DataFrame` you create is an instance of it — everything Week 2, Chapter 1 taught about instances having their own data and methods applies directly here. `df.head()`, `df.describe()`, and every other `df.something()` call is just an instance method being called on your particular `DataFrame` object.

3.1 Series and DataFrame

A **Series** is a single labeled column of data — think of it as a NumPy array with an index attached. A **DataFrame** is a table: multiple Series sharing the same row index, each one a named column.

```
import pandas as pd

s = pd.Series([90, 85, 78], name="score")
print(s)
# 0    90
# 1    85
# 2    78
# Name: score, dtype: int64

df = pd.DataFrame({
    "name": ["Amara", "Leo", "Priya"],
    "score": [90, 85, 78],
})
print(df)
#   name  score
# 0  Amara    90
# 1   Leo    85
# 2  Priya    78
```

Notice the `DataFrame` constructor takes a dict of lists — column name to list of values. This is the same nested-structure pattern (a dict mapping to lists) from Week 1's data structures chapter, just handed to Pandas.

3.2 Loading Data Safely

```
try:
    df = pd.read_csv("students.csv")
except FileNotFoundError:
    print("Couldn't find students.csv — check the file path")
```

Connecting back: this is exactly the specific-exception-handling pattern from Week 2, Chapter 5 — catching `FileNotFoundError` by name instead of a bare `except:`, so a genuinely different problem doesn't get silently misreported as a missing file.

3.3 Inspecting a DataFrame

```
df.head()          # first 5 rows
df.tail(3)         # last 3 rows
df.shape           # (rows, columns) tuple
df.columns         # column names
df.dtypes          # data type of each column
df.info()          # column names, types, and non-null counts in one summary
df.describe()      # count, mean, std, min/max, quartiles for numeric columns
```

3.4 Selecting Data

```
df["score"]                # a single column, returned as a Series
df[["name", "score"]]      # multiple columns, returned as a DataFrame

df.loc[0]                  # row by LABEL (index value)
df.iloc[0]                 # row by POSITION (0-based, like a list)
df.loc[0, "score"]         # a specific cell, by label

df[df["score"] >= 80]      # boolean filtering – same masking idea as NumPy
```

.loc vs .iloc : `.loc` uses the index's actual labels (which are usually, but not always, 0, 1, 2...). `.iloc` always uses position, regardless of what the labels are. If a DataFrame has been filtered or re-sorted, its labels no longer match position order — that's exactly when the difference matters.

Where You'll Actually Use This

Loading a fine-tuning dataset — a JSONL file of prompt/completion pairs almost always gets loaded into a DataFrame first, for inspection and filtering before it's turned into a training format:

```
df = pd.DataFrame([
    {"prompt": "Summarize this article", "completion": "The article discusses...", "quality": 0.9},
    {"prompt": "Translate to French", "completion": "Bonjour", "quality": 0.4},
    {"prompt": "Explain gravity", "completion": "Gravity is a force...", "quality": 0.85},
])
print(df.shape)  # (3, 3)
```

Inspecting a RAG document corpus — checking document count and length distribution before spending time and money embedding any of it:

```
corpus = pd.DataFrame({
    "doc_id": [1, 2, 3],
    "text": ["Short doc.", "A medium-length document about policies.", "A" * 3000],
})
corpus["length"] = corpus["text"].str.len()
print(corpus["length"].describe())
# tells you immediately if you have any documents way too long or too short to be useful chunks
```

Filtering a dataset down to a subset — keeping only the high-quality rows before they become training data:

```
high_quality = df[df["quality"] >= 0.8]
print(high_quality["prompt"].tolist())  # ['Summarize this article', 'Explain gravity']
```

Chapter 4 — Pandas Data Cleaning

By the end of this chapter you should be able to

- Find and handle missing data
- Find and remove duplicate rows
- Convert column types safely and clean up string columns
- Apply a custom function or lambda across a column

4.1 Finding Missing Data

```
df.isna()           # a same-shaped DataFrame of True/False
df.isna().sum()     # count of missing values PER COLUMN – the most useful first check
```

4.2 Handling Missing Data

```
df.dropna()          # drop any row with at least one missing value
df.dropna(subset=["score"]) # only drop rows missing THIS column
df.dropna(axis=1)     # drop COLUMNS with any missing value instead of rows

df["score"].fillna(0) # replace missing values with a fixed value
df["score"].fillna(df["score"].mean()) # replace with the column's mean – very common
df["score"].ffill()   # forward-fill: carry the last valid value forward
```

Think before you drop or fill. Dropping rows loses data — if 30% of a column is missing, dropping every affected row may bias what's left. Filling with the mean is convenient but quietly distorts the column's real distribution. There's no universally correct choice; the right one depends on *why* the data is missing, which is a judgment call, not a one-liner.

4.3 Duplicates

```
df.duplicated()      # True/False per row – is this row a repeat of an earlier one?
df.duplicated().sum() # count of duplicate rows
df.drop_duplicates()  # remove them, keeping the first occurrence by default
df.drop_duplicates(subset=["name"]) # only consider THIS column when deciding "duplicate"
```

4.4 Type Conversion

```
df["score"] = df["score"].astype(float)
df["score"] = pd.to_numeric(df["score"], errors="coerce") # invalid values become NaN, not a crash
df["signup_date"] = pd.to_datetime(df["signup_date"], errors="coerce")
```

Connecting back: `errors="coerce"` is Pandas' built-in way of doing what Week 1's type-casting chapter warned about manually — turning a bad conversion into a clear, checkable value (`NaN`) instead of crashing the whole pipeline on one dirty row.

4.5 Cleaning String Columns

```
df["name"] = df["name"].str.strip() # remove leading/trailing whitespace
df["name"] = df["name"].str.lower() # normalize case
df["name"] = df["name"].str.replace("_", " ") # find-and-replace within every value
```

The `.str` accessor is what makes this work — it applies a string method to every value in the column at once, the same vectorized spirit as NumPy math, just for text.

4.6 Renaming & Restructuring

```
df.rename(columns={"nm": "name", "scr": "score"}) # rename specific columns
df.columns = ["name", "score"]                   # or replace all column names at once
df.drop(columns=["unused_column"])                 # drop a column entirely
```

4.7 Applying Custom Logic — `.apply()` and `.map()`

```
# .apply() on a Series — runs a function on every value
df["passed"] = df["score"].apply(lambda x: "Yes" if x >= 60 else "No")

# .map() on a Series — similar, often used for value substitution via a dict
grade_labels = {90: "A", 85: "B", 78: "C"}
df["letter"] = df["score"].map(grade_labels)

# .apply() on a whole DataFrame with axis=1 — runs a function per ROW
df["summary"] = df.apply(lambda row: f'{row["name"]}: {row["score"]}', axis=1)
```

4.8 Common Pitfalls

- **SettingWithCopyWarning** : happens when you modify a DataFrame that might be a view of another one rather than an independent copy. If you intend to keep working with a filtered subset, be explicit: `subset = df[df["score"] > 80].copy()` .
- **Chained indexing**: `df[df["score"] > 80]["score"] = 100` may silently fail to update the original DataFrame. Use `.loc` for combined filtering-and-assignment instead: `df.loc[df["score"] > 80, "score"] = 100` .
- **Forgetting most Pandas operations return a new object** rather than modifying in place — `df.dropna()` alone doesn't change `df` ; you need `df = df.dropna()` or `inplace=True` .

Where You'll Actually Use This

Deduplicating a fine-tuning dataset — near-identical training examples (common when scraping or combining sources) can bias a fine-tuned model toward repeated patterns:

```
training_data = pd.DataFrame({
    "prompt": ["What is Python?", "What is Python?", "What is a variable?"],
    "completion": ["A programming language.", "A programming language.", "A named storage location."],
})
deduped = training_data.drop_duplicates()
print(deduped.shape)  # (2, 2) — the exact-duplicate row is gone
```

Cleaning scraped text before embedding it — garbage in, garbage vectors out:

```
raw = pd.DataFrame({"text": [" Hello World ", "", "Valid document text.", None]})
raw["text"] = raw["text"].str.strip()
clean = raw[raw["text"].notna() & (raw["text"] != "")]
print(clean["text"].tolist())  # ['Hello World', 'Valid document text.']
```

Standardizing inconsistent labels — collapsing messy category values into one consistent label before they become training data:

```
df = pd.DataFrame({"country": ["USA", "U.S.", "United States", "Canada"]})
country_map = {"USA": "United States", "U.S.": "United States", "United States": "United States"}
df["country"] = df["country"].map(country_map).fillna(df["country"])
print(df["country"].tolist())  # ['United States', 'United States', 'United States', 'Canada']
```

Chapter 5 — Vectors & Similarity

By the end of this chapter you should be able to

- Explain what a vector and an embedding are, and how they relate
- Compute a vector's magnitude and the dot product of two vectors
- Compute cosine similarity and Euclidean distance, and explain when to use each
- Explain where cosine similarity fits in a semantic search or RAG pipeline

Why this chapter matters more than its length suggests: cosine similarity is the actual math running underneath semantic search, RAG retrieval, and recommendation — every time an AI system finds "the most relevant chunk" or "the most similar item," it is almost always computing exactly the calculation in this chapter. Everything else in this course that involves "finding similar things" reduces to what you're about to learn here.

5.1 What a Vector Is, and What an Embedding Is

A **vector** is just an ordered list of numbers. In an AI engineering context, that list of numbers represents something — a row of numeric features, an image's pixel values flattened out, or, most importantly for this course, a piece of text.

An **embedding** is a specific, very common kind of vector: the output of a model that has read a piece of text (a word, a sentence, a whole document) and converted its *meaning* into a fixed-length list of numbers — typically hundreds or thousands of dimensions. Two pieces of text with similar meaning end up with embedding vectors that point in similar directions, even if they don't share a single word in common. That single property — meaning becomes distance — is the foundation of semantic search.

```
# conceptually, this is what an embedding call returns (illustrative, not a real API)
# embed("The cat sat on the mat")      -> [0.02, -0.31, 0.88, ..., 0.14] (e.g. 1536 numbers)
# embed("A feline rested on the rug")   -> [0.03, -0.29, 0.85, ..., 0.11] (very close to the first!)
# embed("Stock prices rose sharply")    -> [-0.71, 0.44, -0.02, ..., 0.90] (far from both)
```

Connecting back: a single row of a Pandas DataFrame (once it's all-numeric and cleaned) is a vector in exactly this sense — this is why Chapter 4's cleaning work matters before this chapter's math can run on it. Dirty or missing values break vector math immediately, and an embedding model fed messy, uncleaned text produces messy, unreliable embeddings.

To build intuition, the rest of this chapter uses small, simple 3-number vectors you can compute by hand — the exact same operations apply identically whether a vector has 3 numbers or 3,000.

```
import numpy as np

student_a = np.array([90, 85, 78]) # e.g., scores on 3 tests
student_b = np.array([88, 80, 75])
```

5.2 Basic Vector Operations

```
print(student_a + student_b) # [178 165 153] - elementwise addition
print(student_a - student_b) # [2 5 3]
print(student_a * 2)         # [180 170 156] - scalar multiplication
```

5.3 Dot Product

The dot product multiplies corresponding elements and sums the results — a single number summarizing how two vectors relate.

```
dot = np.dot(student_a, student_b)
print(dot) # 90*88 + 85*80 + 78*75 = 20570
```

5.4 Magnitude (Norm)

A vector's magnitude is its length — how far it is from the origin.

```
magnitude = np.linalg.norm(student_a)
print(magnitude) # sqrt(90^2 + 85^2 + 78^2) ≈ 146.32
```

5.5 Euclidean Distance

The straight-line distance between two vectors — smaller means more similar.

```
distance = np.linalg.norm(student_a - student_b)
print(distance)    # ≈ 6.16
```

5.6 Cosine Similarity

Cosine similarity measures the *angle* between two vectors, ignoring their magnitude entirely — it asks "do these point in the same direction?" rather than "are these the same size?" It ranges from -1 (opposite) to 1 (identical direction), with 0 meaning unrelated.

```
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(cosine_similarity(student_a, student_b))    # ≈ 0.9998 – nearly identical direction
```

Why cosine over Euclidean distance so often in AI: for text embeddings specifically, the *direction* a vector points carries the meaning, while its magnitude can vary for reasons that don't matter (like text length). Cosine similarity ignores that irrelevant magnitude difference; Euclidean distance does not. This is exactly why cosine similarity — not Euclidean distance — is the standard, default choice almost everywhere embeddings are compared.

5.7 The Practical Tool: scikit-learn

Writing cosine similarity by hand (above) is worth doing once to understand it — in practice, you'll use a tested library implementation:

```
from sklearn.metrics.pairwise import cosine_similarity

vectors = np.array([student_a, student_b])
print(cosine_similarity(vectors))
# [[1.         0.9998...]
#  [0.9998...  1.         ]]  – a full similarity matrix, every vector compared to every other
```

5.8 Semantic Search & RAG: Where This Math Actually Lives

A Retrieval-Augmented Generation (RAG) system — or any semantic search feature — follows the same basic pipeline:

1. **Chunk** your source documents into manageable pieces (a paragraph, a section)
2. **Embed** every chunk into a vector using an embedding model
3. **Store** those vectors in a vector database (e.g., Pinecone, Chroma, FAISS, or even a plain NumPy array for small datasets)
4. **Embed the user's query** the same way, into the same vector space
5. **Retrieve** the stored chunks whose vectors have the highest cosine similarity to the query vector
6. Hand those retrieved chunks to an LLM as context to answer the user's question

```
# a tiny, from-scratch illustration of steps 3-5, using toy vectors
# in a real system, 'doc_vectors' would come from an embedding model, and there
# might be millions of them, held in a proper vector database instead of an array

doc_vectors = np.array([
    [0.02, -0.31, 0.88],    # "The cat sat on the mat"
    [0.03, -0.29, 0.85],    # "A feline rested on the rug"
    [-0.71, 0.44, -0.02],   # "Stock prices rose sharply"
])
doc_texts = ["The cat sat on the mat", "A feline rested on the rug", "Stock prices rose sharply"]

query_vector = np.array([0.01, -0.30, 0.86])    # embedding of "Where did the cat rest?"

similarities = cosine_similarity(doc_vectors, query_vector.reshape(1, -1)).flatten()
best_match = doc_texts[np.argmax(similarities)]
print(best_match)    # "The cat sat on the mat" – highest cosine similarity to the query
```

The whole point of this chapter, in one sentence: a vector database is, at its core, a system built to do exactly this `cosine_similarity + argmax` lookup extremely fast across millions of vectors instead of three.

Where You'll Actually Use This

RAG retrieval & semantic search — finding which document chunks are relevant to a user's question, "search by meaning" instead of exact keyword matching. This is the single most common use of this math in AI engineering — see the full worked example in 5.8 above.

Recommendation — "users similar to you also liked..." by comparing user or item vectors the same way:

```
user_vectors = np.array([
    [5, 1, 0],    # user A's ratings: [action, romance, comedy]
    [4, 1, 1],    # user B – similar taste to A
    [0, 5, 1],    # user C – very different taste
])
target_user = user_vectors[0].reshape(1, -1)

similarities = cosine_similarity(user_vectors, target_user).flatten()
print(similarities)           # [1.    0.971 0.192] – user B is the closest match to user A
print(np.argsort(similarities)[::-1][1])  # 1 – index of the most similar OTHER user (skipping self at index 0)
```

Deduplication & clustering — flagging near-duplicate support tickets or training examples by vector distance, rather than requiring an exact text match:

```
ticket_vectors = np.array([
    [0.9, 0.1, 0.2],    # "My order hasn't arrived"
    [0.88, 0.12, 0.19], # "Where is my order?" – nearly identical meaning
    [0.1, 0.9, 0.3],    # "How do I reset my password?" – unrelated
])
sims = cosine_similarity(ticket_vectors)
duplicate_pairs = np.argwhere((sims > 0.95) & (sims < 1.0))
print(duplicate_pairs)  # flags tickets 0 and 1 as likely duplicates of each other
```

Chapter 6 — Train/Test Splitting

By the end of this chapter you should be able to

- Explain why data must be split before training a model
- Use `train_test_split` correctly, including `random_state` and `stratify`
- Explain data leakage and how splitting order prevents it

Why this still matters in an AI engineering context: this chapter uses classic supervised-learning examples (features and labels) because that's the clearest way to learn the mechanics — but the underlying idea, an honest held-out set you never touch until final evaluation, is exactly what you'll reach for later to evaluate a fine-tuned model, compare two prompt strategies, or measure whether a RAG pipeline's retrieval quality actually improved. "Don't evaluate on what you tuned on" is the same rule whether "tuning" means adjusting model weights or adjusting a prompt.

6.1 Why Split at All

A model evaluated on the same data it was trained on will look deceptively good — it can effectively "memorize" the training data rather than learn a pattern that generalizes. This is called **overfitting**. Splitting data into a training set and a held-out test set gives an honest estimate of how the model performs on data it has never seen.

6.2 `train_test_split`

```
from sklearn.model_selection import train_test_split
import numpy as np

X = np.arange(20).reshape(10, 2)  # 10 rows of "features"
y = np.array([0, 1, 0, 1, 0, 1, 0, 1, 0, 1])  # 10 corresponding labels

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, X_test.shape)  # (8, 2) (2, 2) – 80/20 split
```

Parameter	What it controls
<code>test_size</code>	Fraction (e.g., 0.2 = 20%) or exact count held out for testing
<code>random_state</code>	Seeds the random shuffle — set it to a fixed number for reproducible splits across runs
<code>stratify</code>	Keeps the same class proportions in both the train and test sets (crucial for imbalanced classification data)

```
# stratified split – keeps the label balance consistent in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

6.3 Train / Validation / Test — A Three-Way Split

In practice, you often need a **third** slice: a validation set used to tune a model while developing it, keeping the test set completely untouched until the very end for a final, unbiased check.

```
# split twice: first carve out test, then split the remainder into train/validation
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42)
# 0.25 of the remaining 80% = 20% of the original total – so this yields a 60/20/20 split overall
```

6.4 Data Leakage

The order of operations matters. Any preprocessing that learns something from the data — computing a mean to fill missing values, scaling features, fitting a vectorizer — must be fit on the **training set only**, then applied to the test set using those same training-derived values. Fitting on the full dataset before splitting leaks information from the test set into training, making your evaluation overly optimistic and unreliable once the model meets truly new data.

```
# WRONG — fills missing values using stats from the WHOLE dataset, including test data
df["score"] = df["score"].fillna(df["score"].mean())
X_train, X_test, y_train, y_test = train_test_split(df, y)

# RIGHT — split first, then fit only on the training portion
X_train, X_test, y_train, y_test = train_test_split(df, y)
train_mean = X_train["score"].mean()
X_train["score"] = X_train["score"].fillna(train_mean)
X_test["score"] = X_test["score"].fillna(train_mean) # reuse the TRAIN mean, not a new test mean
```

Connecting back: validating that a split actually adds up correctly is a natural fit for an `assert`, straight from Week 2, Chapter 5 — e.g., `assert len(X_train) + len(X_test) == len(X)` as a quick sanity check during development, not a substitute for real error handling in production code.

Where You'll Actually Use This

Evaluating a fine-tuned model honestly — holding out examples the model never saw during fine-tuning, so accuracy numbers reflect real performance, not memorization:

```
examples = pd.DataFrame({
    "prompt": [f"question {i}" for i in range(100)],
    "label": [i % 2 for i in range(100)],
})

train_set, eval_set = train_test_split(examples, test_size=0.2, random_state=42)
print(len(train_set), len(eval_set)) # 80 20 — fine-tune on train_set, score only on eval_set
```

Comparing prompt strategies fairly — running two approaches against the exact same held-out questions instead of eyeballing a handful of examples:

```
_, eval_questions = train_test_split(examples, test_size=0.2, random_state=42)

def score_strategy(strategy_fn, questions):
    correct = sum(strategy_fn(q) == label for q, label in zip(questions["prompt"], questions["label"]))
    return correct / len(questions)

# both strategies get scored against the IDENTICAL eval_questions — a fair comparison
# accuracy_a = score_strategy(strategy_a, eval_questions)
# accuracy_b = score_strategy(strategy_b, eval_questions)
```

Building a labeled eval set for an LLM-based classifier — splitting hand-labeled examples before using train for few-shot examples and test purely for scoring:

```
labeled_data = pd.DataFrame({
    "text": [f"support ticket {i}" for i in range(50)],
    "category": (["billing", "technical"] * 25),
})

few_shot_pool, scoring_set = train_test_split(labeled_data, test_size=0.3, random_state=42, stratify=labeled_data["category"])
print(scoring_set["category"].value_counts()) # 8 billing, 7 technical — stratify preserves the original 50/50 proportion
```

How It All Connects — and Before You Move On

This guide sits on top of everything from Weeks 1 and 2, not beside it:

- **Week 1's** list-of-dicts and nested data structures are exactly the shape a Pandas DataFrame is built from.
- **Week 1's** type casting warnings are what `pd.to_numeric(..., errors="coerce")` handles automatically at scale.
- **Week 2's** higher-order functions and first-class functions are the entire reason lambda syntax exists and is used constantly with `.apply()`.
- **Week 2's** specific-exception handling is exactly how you should wrap `pd.read_csv()` and any other data-loading step.
- **Week 2's** OOP chapter is why a DataFrame's dozens of methods (`.head()`, `.dropna()`, `.apply()` ...) all make sense as instance methods on an object, not disconnected magic.
- **Week 2's** `assert` statement is a natural fit for sanity-checking a train/test split during development.

And it sets up everything still ahead:

- The vector math in Chapter 5 is the literal retrieval mechanism you'll use when this course reaches **RAG pipelines** — chunking documents, embedding them, and retrieving the right ones is Chapter 5's cosine similarity at production scale.
- The cleaning skills in Chapter 4 are exactly what you'll apply to a **fine-tuning dataset** before it's usable — deduplication, consistent formatting, and filtering bad examples matter enormously for fine-tuning quality.
- The train/test discipline in Chapter 6 is exactly how you'll **evaluate a fine-tuned model or compare prompt/RAG strategies** fairly, once you're building systems worth evaluating rigorously.

Self-Check

- ☐ I can write a lambda and explain when to use one instead of a full `def`
- ☐ I can use lambda with `sorted()`, `map()`, and `filter()`
- ☐ I can create, index, slice, and reshape a NumPy array
- ☐ I can explain broadcasting and predict when two shapes are compatible
- ☐ I can write vectorized NumPy code instead of a Python loop, and explain why it's faster
- ☐ I know the difference between a Series and a DataFrame, and how both relate to NumPy arrays
- ☐ I can load a CSV safely with exception handling, and inspect a DataFrame with `.head()`, `.info()`, and `.describe()`
- ☐ I can find and handle missing data with `.isna()`, `.dropna()`, and `.fillna()`
- ☐ I can find and remove duplicates, and convert column types safely
- ☐ I can use `.apply()` and `.map()` with a lambda to transform a column
- ☐ I know the difference between `.loc` and `.iloc`, and how to avoid `SettingWithCopyWarning`
- ☐ I can explain what an embedding is and how it relates to a vector
- ☐ I can compute a vector's magnitude, dot product, cosine similarity, and Euclidean distance
- ☐ I can explain the chunk → embed → store → retrieve pipeline and where cosine similarity fits in it
- ☐ I can explain why cosine similarity is preferred over Euclidean distance for embeddings
- ☐ I can explain why data must be split before training a model, and what overfitting means
- ☐ I can use `train_test_split` with `test_size`, `random_state`, and `stratify`
- ☐ I can explain data leakage and why preprocessing must be fit on the training set only